

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import plotly.graph_objects as go
from scipy.signal import find_peaks
from scipy.stats import sem
```

```
In [2]: df = pd.read_csv("D:\IITB\ppt\DS 203\e7-htr-currennt.csv")
```

```
In [3]: records = df[['Current', 'day', 'month', 'year', 'hour']]
```

```
In [4]: records.head
```

```
Out[4]: <bound method NDFrame.head of
0      0.0  23  12  2018    5
1      0.0  23  12  2018    5
2      0.0  23  12  2018    5
3      0.0  23  12  2018    5
4      0.0  23  12  2018    5
...     ...  ...  ...  ...  ...
81721    0.0   4  10  2019   23
81722    0.0   4  10  2019   23
81723    0.0   4  10  2019   23
81724    0.0   4  10  2019   23
81725    0.0   4  10  2019   23
```

[81726 rows x 5 columns]>

```
In [5]: good = pd.DataFrame()
bad = pd.DataFrame()
missing = pd.DataFrame()
```

```
In [6]: #Good day example
l = df.loc[(records['day'] == 6) & (records['month'] == 8)]
l
```

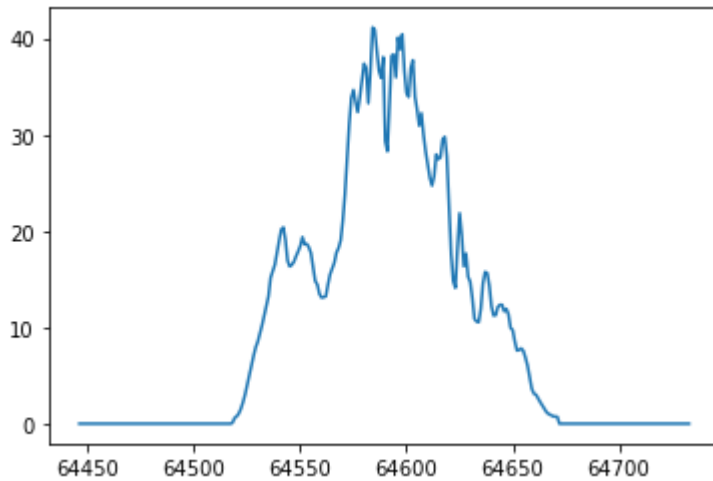
```
Out[6]:
```

|              | Timestamp        | Current | day | month | year | hour |
|--------------|------------------|---------|-----|-------|------|------|
| <b>64446</b> | 06-08-2019 00:00 | 0.1     | 6   | 8     | 2019 | 0    |
| <b>64447</b> | 06-08-2019 00:05 | 0.1     | 6   | 8     | 2019 | 0    |
| <b>64448</b> | 06-08-2019 00:10 | 0.1     | 6   | 8     | 2019 | 0    |
| <b>64449</b> | 06-08-2019 00:15 | 0.1     | 6   | 8     | 2019 | 0    |
| <b>64450</b> | 06-08-2019 00:20 | 0.1     | 6   | 8     | 2019 | 0    |
| ...          | ...              | ...     | ... | ...   | ...  | ...  |
| <b>64729</b> | 06-08-2019 23:35 | 0.1     | 6   | 8     | 2019 | 23   |
| <b>64730</b> | 06-08-2019 23:40 | 0.1     | 6   | 8     | 2019 | 23   |
| <b>64731</b> | 06-08-2019 23:45 | 0.1     | 6   | 8     | 2019 | 23   |
| <b>64732</b> | 06-08-2019 23:50 | 0.1     | 6   | 8     | 2019 | 23   |
| <b>64733</b> | 06-08-2019 23:55 | 0.1     | 6   | 8     | 2019 | 23   |

288 rows x 6 columns

```
In [7]: plt.plot(l['Current'])
```

```
Out[7]: [<matplotlib.lines.Line2D at 0x219de9f0c40>]
```



```
In [8]: #Bad day example
k = df.loc[(records['day'] == 5) & (records['month'] == 8)]
k
```

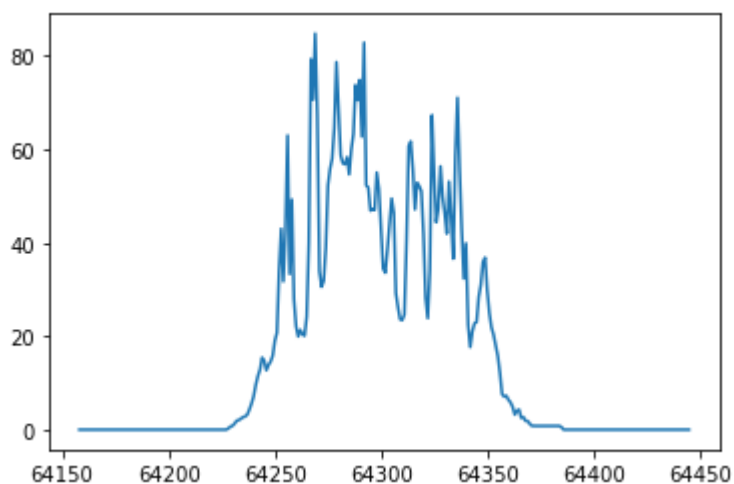
```
Out[8]:
```

|              | Timestamp        | Current | day | month | year | hour |
|--------------|------------------|---------|-----|-------|------|------|
| <b>64158</b> | 05-08-2019 00:00 | 0.1     | 5   | 8     | 2019 | 0    |
| <b>64159</b> | 05-08-2019 00:05 | 0.1     | 5   | 8     | 2019 | 0    |
| <b>64160</b> | 05-08-2019 00:10 | 0.1     | 5   | 8     | 2019 | 0    |
| <b>64161</b> | 05-08-2019 00:15 | 0.1     | 5   | 8     | 2019 | 0    |
| <b>64162</b> | 05-08-2019 00:20 | 0.1     | 5   | 8     | 2019 | 0    |
| ...          | ...              | ...     | ... | ...   | ...  | ...  |
| <b>64441</b> | 05-08-2019 23:35 | 0.1     | 5   | 8     | 2019 | 23   |
| <b>64442</b> | 05-08-2019 23:40 | 0.1     | 5   | 8     | 2019 | 23   |
| <b>64443</b> | 05-08-2019 23:45 | 0.1     | 5   | 8     | 2019 | 23   |
| <b>64444</b> | 05-08-2019 23:50 | 0.1     | 5   | 8     | 2019 | 23   |
| <b>64445</b> | 05-08-2019 23:55 | 0.1     | 5   | 8     | 2019 | 23   |

288 rows × 6 columns

```
In [9]: plt.plot(k['Current'])
```

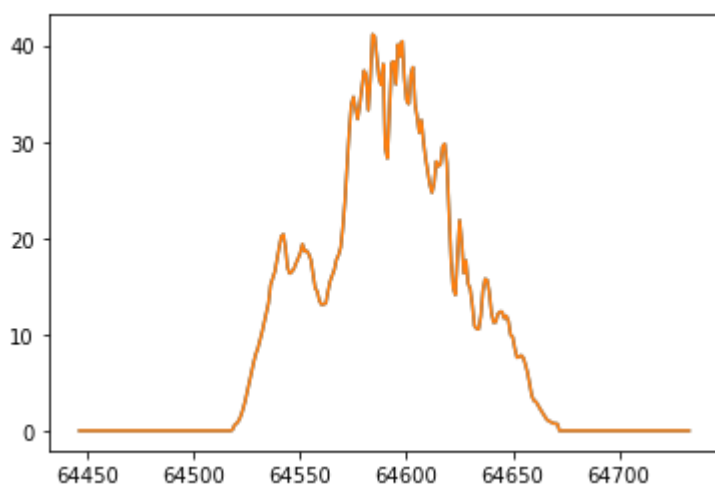
```
Out[9]: [<matplotlib.lines.Line2D at 0x219df5b8310>]
```



```
In [10]: #For a Good day no outliers
Q1 = l['Current'].quantile(0.25)
Q3 = l['Current'].quantile(0.75)
IQR = Q3 - Q1
lb = Q1 - 1.5 * IQR
ub = Q3 + 1.5 * IQR

l_cleaned = l[(l['Current'] >= lb) & (l['Current'] <= ub)]
plt.plot(l['Current'])
plt.plot(l_cleaned['Current'])
```

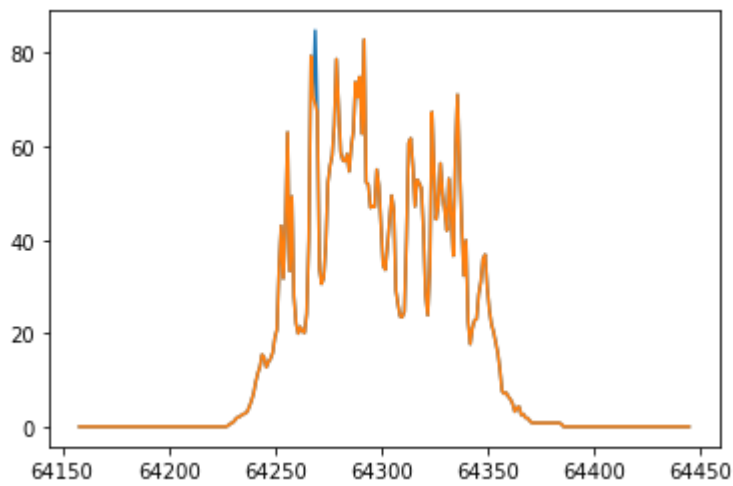
Out[10]: [matplotlib.lines.Line2D at 0x219df622670]



```
In [11]: #Outliers Detected in Bad data
Q1 = k['Current'].quantile(0.25)
Q3 = k['Current'].quantile(0.75)
IQR = Q3 - Q1
lb = Q1 - 1.5 * IQR
ub = Q3 + 1.5 * IQR

k_cleaned = k[(k['Current'] >= lb) & (k['Current'] <= ub)]
plt.plot(k['Current'])
plt.plot(k_cleaned['Current'])
```

Out[11]: [matplotlib.lines.Line2D at 0x219df676ee0]



```
In [12]: #For a Bad Data plotting peaks and valleys
fig = go.Figure()
fig.add_trace(go.Scatter(y=k['Current'],
                        mode='lines+markers',
                        name='Original Plot'))

peak_indices = find_peaks(k['Current'])[0]
valley_indices = find_peaks(-k['Current'])[0]

c = np.zeros(1)
d = np.zeros(1)
e = np.zeros(1)

for i,r in k.iterrows():
    d = np.append(d,r[1])
d = np.delete(d,0)

for j in peak_indices:
    c = np.append(c,d[j])
c = np.delete(c,0)

for j in valley_indices:
    e = np.append(e,d[j])
e = np.delete(e,0)

fig.add_trace(go.Scatter(x=peak_indices, y = c,
                        mode='markers',
                        marker=dict(
                            size=8,
                            color='red',
                            symbol='cross'
                        ),
                        name='Detected Peaks'))

fig.add_trace(go.Scatter(x=valley_indices, y = e,
                        mode='markers',
                        marker=dict(
                            size=8,
                            color='green',
                            symbol='cross'
                        ),
                        name='Detected Valleys'))
```

```
In [13]: #For Good Data plotting peaks and valleys
fig = go.Figure()
fig.add_trace(go.Scatter(y=l['Current'],
```

```

mode='lines+markers',
name='Original Plot'))

peak_indices = find_peaks(l['Current'])[0]
valley_indices = find_peaks(-l['Current'])[0]

c = np.zeros(1)
d = np.zeros(1)
e = np.zeros(1)

for i,r in l.iterrows():
    d = np.append(d,r[1])
d = np.delete(d,0)

for j in peak_indices:
    c = np.append(c,d[j])
c = np.delete(c,0)

for j in valley_indices:
    e = np.append(e,d[j])
e = np.delete(e,0)

fig.add_trace(go.Scatter(x=peak_indices, y = c,
    mode='markers',
    marker=dict(
        size=8,
        color='red',
        symbol='cross'
    ),
    name='Detected Peaks'))

fig.add_trace(go.Scatter(x=valley_indices, y = e,
    mode='markers',
    marker=dict(
        size=8,
        color='green',
        symbol='cross'
    ),
    name='Detected Valleys'))

```

In [14]: *#There is Large variation in Bad data and Less to moderate variation in Good data*

```

In [34]: g = np.zeros(1)
b = np.zeros(1)
m = np.zeros(1)

```

```

In [35]: for k in [4,6,9]:
    for i in range(1,31):
        l = df.loc[(records['day'] == i) & (records['month'] == k)]

        tv = 0
        count = 0
        p = 0
        for j,r in l.iterrows():
            count = count + 1
            if count > 1:
                diff = abs(r[1]-p)
                tv += diff
                p = r[1]

        print(i , tv)

```

```

    if tv < 380 and tv > 0:
        g = np.append(g,i)
        good = pd.concat([good,1])
    elif tv == 0:
        m = np.append(m,i)
        missing = pd.concat([missing,1])
    elif tv > 380:
        b = np.append(b,i)
        bad = pd.concat([bad,1])

for k in [1,3,5,7,8]:
    for i in range(1,32):
        l = df.loc[(records['day'] == i) & (records['month'] == k)]

        tv = 0
        count = 0
        p = 0
        for j,r in l.iterrows():
            count = count + 1
            if count > 1:
                diff = abs(r[1]-p)
                tv += diff
                p = r[1]

        print(i , tv)

        if tv < 380 and tv > 0:
            g = np.append(g,i)
            good = pd.concat([good,1])
        elif tv == 0:
            m = np.append(m,i)
            missing = pd.concat([missing,1])
        elif tv > 380:
            b = np.append(b,i)
            bad = pd.concat([bad,1])

for k in [2,3]:
    for i in range(1,29):
        l = df.loc[(records['day'] == i) & (records['month'] == k)]

        tv = 0
        count = 0
        p = 0
        for j,r in l.iterrows():
            count = count + 1
            if count > 1:
                diff = abs(r[1]-p)
                tv += diff
                p = r[1]

        print(i , tv)

        if tv < 380 and tv > 0:
            g = np.append(g,i)
            good = pd.concat([good,1])
        elif tv == 0:
            m = np.append(m,i)
            missing = pd.concat([missing,1])
        elif tv > 380:
            b = np.append(b,i)
            bad = pd.concat([bad,1])

for i in range(1,5):
    l = df.loc[(records['day'] == i) & (records['month'] == 10)]

```

```

tv = 0
count = 0
p = 0
for j,r in l.iterrows():
    count = count + 1
    if count > 1:
        diff = abs(r[1]-p)
        tv += diff
        p = r[1]

print(i , tv)

if tv < 380 and tv > 0:
    g = np.append(g,i)
    good = pd.concat([good,1])
elif tv == 0:
    m = np.append(m,i)
    missing = pd.concat([missing,1])
elif tv > 380:
    b = np.append(b,i)
    bad = pd.concat([bad,1])

for i in range(23,32):
    l = df.loc[(records['day'] == i) & (records['month'] == 12)]

    tv = 0
    count = 0
    p = 0
    for j,r in l.iterrows():
        count = count + 1
        if count > 1:
            diff = abs(r[1]-p)
            tv += diff
            p = r[1]

    print(i , tv)

    if tv < 380 and tv > 0:
        g = np.append(g,i)
        good = pd.concat([good,1])
    elif tv == 0:
        m = np.append(m,i)
        missing = pd.concat([missing,1])
    elif tv > 380:
        b = np.append(b,i)
        bad = pd.concat([bad,1])

```

1 170.08000000000013  
2 318.34999999999996  
3 285.88999999999993  
4 183.27000000000007  
5 379.22999999999999  
6 497.48999999999995  
7 502.50999999999993  
8 264.85999999999996  
9 307.06999999999995  
10 332.929999999999967  
11 204.64000000000004  
12 228.86000000000004  
13 595.85999999999999  
14 173.75999999999996  
15 167.69999999999996  
16 177.64999999999998  
17 189.36999999999986  
18 320.83999999999986  
19 474.3700000000001  
20 399.9200000000001  
21 573.8500000000001  
22 568.2499999999997  
23 193.78  
24 213.16999999999996  
25 136.29999999999987  
26 280.85999999999996  
27 216.23000000000005  
28 348.8200000000001  
29 250.86999999999995  
30 307.95999999999987  
1 754.3100000000001  
2 1238.3200000000002  
3 1206.8299999999995  
4 395.0300000000002  
5 182.90000000000006  
6 552.9699999999997  
7 476.3500000000002  
8 335.83000000000004  
9 193.72999999999993  
10 368.7300000000001  
11 564.95  
12 354.23999999999999  
13 622.6700000000002  
14 679.49999999999999  
15 744.26999999999999  
16 468.4200000000002  
17 296.62  
18 1149.6899999999998  
19 96.40999999999997  
20 868.1400000000002  
21 419.61999999999999  
22 164.14999999999995  
23 503.39000000000016  
24 0.0  
25 0.0  
26 0.0  
27 1348.4700000000005  
28 465.97999999999999  
29 356.16000000000014  
30 416.83000000000004  
1 801.8499999999997  
2 665.75  
3 293.71000000000015  
4 1757.39



5 2128.5699999999993  
6 1179.7399999999993  
7 2021.48  
8 1444.8199999999995  
9 2561.13  
10 691.2299999999999  
11 564.6999999999999  
12 448.05999999999983  
13 347.86  
14 1095.8099999999993  
15 970.1699999999997  
16 740.1500000000002  
17 840.1799999999998  
18 1382.0  
19 1117.32  
20 1058.5699999999993  
21 1024.7499999999998  
22 989.2600000000001  
23 1045.9699999999998  
24 462.94999999999976  
25 0.0  
26 0.1  
27 1259.8200000000004  
28 633.0100000000001  
29 30.81  
30 32.27000000000001  
1 309.2900000000001  
2 148.73000000000002  
3 0.0  
4 0.0  
5 0.0  
6 392.08  
7 179.44  
8 0.0  
9 0.0  
10 0.0  
11 0.0  
12 0.0  
13 0.0  
14 0.0  
15 0.0  
16 0.0  
17 0.0  
18 0.0  
19 0.0  
20 0  
21 0.0  
22 0.0  
23 0.15  
24 875.7099999999998  
25 163.44999999999996  
26 529.4400000000006  
27 313.66  
28 292.90000000000003  
29 168.67000000000007  
30 1033.2900000000004  
31 1321.0100000000002  
1 0.0  
2 325.95000000000005  
3 161.15999999999997  
4 593.4900000000005  
5 308.99999999999994  
6 617.77  
7 243.71

8 1126.5200000000007  
9 0.0  
10 609.2800000000001  
11 109.74  
12 352.65999999999985  
13 281.3399999999998  
14 271.0399999999999  
15 161.20000000000002  
16 155.95999999999998  
17 329.97999999999985  
18 351.2400000000001  
19 169.79999999999995  
20 392.09999999999997  
21 458.6999999999999  
22 603.6799999999998  
23 303.43999999999977  
24 155.23999999999992  
25 423.2400000000002  
26 173.69  
27 753.7099999999999  
28 451.36999999999983  
29 314.39999999999975  
30 310.54999999999995  
31 0.0  
1 317.6500000000001  
2 405.7599999999999  
3 381.4500000000001  
4 237.56999999999985  
5 563.5399999999998  
6 523.0300000000002  
7 443.6300000000002  
8 445.3299999999998  
9 219.90000000000026  
10 295.31  
11 406.41000000000025  
12 446.16000000000014  
13 318.74999999999994  
14 428.26  
15 220.96999999999997  
16 555.8800000000001  
17 384.59000000000015  
18 361.09000000000003  
19 678.7299999999993  
20 440.44000000000005  
21 388.5700000000001  
22 345.2099999999998  
23 178.73000000000008  
24 204.84000000000003  
25 427.5000000000001  
26 345.79000000000013  
27 502.31000000000006  
28 185.95999999999995  
29 213.33000000000007  
30 451.60000000000025  
31 526.0200000000001  
1 288.0399999999999  
2 262.36  
3 547.52  
4 1184.7600000000002  
5 450.67999999999995  
6 463.60999999999984  
7 1089.7700000000002  
8 1447.6599999999999  
9 713.9999999999994

10 946.44  
11 969.4700000000003  
12 1109.0499999999997  
13 605.04  
14 951.0099999999996  
15 808.6399999999999  
16 575.97  
17 1527.0799999999997  
18 599.2399999999999  
19 672.7  
20 552.8  
21 0.0  
22 1094.1999999999996  
23 1352.49  
24 1039.7900000000004  
25 622.2599999999996  
26 187.07999999999996  
27 144.36000000000004  
28 117.37999999999995  
29 151.69000000000014  
30 609.2999999999998  
31 1418.2499999999989  
1 675.7100000000002  
2 95.71000000000005  
3 173.39  
4 633.6299999999998  
5 915.0599999999998  
6 204.04000000000002  
7 158.86999999999992  
8 680.57  
9 827.83  
10 1238.7899999999995  
11 628.4  
12 0.0  
13 302.66999999999996  
14 396.76000000000016  
15 791.21  
16 576.1700000000001  
17 777.4799999999997  
18 875.1500000000001  
19 878.9700000000001  
20 571.0799999999995  
21 793.7700000000003  
22 446.91999999999973  
23 420.28000000000014  
24 0.0  
25 0.0  
26 0.0  
27 136.03  
28 1033.0599999999997  
29 2280.04  
30 1672.3299999999997  
31 451.4799999999999  
1 231.73  
2 355.07999999999987  
3 369.25  
4 563.99  
5 178.48000000000008  
6 160.43000000000004  
7 502.7199999999997  
8 738.43  
9 384.4500000000003  
10 267.18000000000006  
11 1056.8999999999994

```

12 537.53000000000001
13 430.10000000000002
14 332.890000000000016
15 325.20999999999998
16 442.93999999999998
17 694.01
18 993.77999999999997
19 905.56000000000004
20 171.31999999999994
21 0.0
22 0.0
23 0.0
24 0.0
25 0.0
26 0.0
27 0.0
28 0.0
1 0.0
2 325.95000000000005
3 161.15999999999997
4 593.49000000000005
5 308.99999999999994
6 617.77
7 243.71
8 1126.52000000000007
9 0.0
10 609.28000000000001
11 109.74
12 352.659999999999985
13 281.33999999999998
14 271.03999999999999
15 161.20000000000002
16 155.95999999999998
17 329.979999999999985
18 351.24000000000001
19 169.79999999999995
20 392.09999999999997
21 458.69999999999999
22 603.67999999999998
23 303.439999999999977
24 155.23999999999992
25 423.24000000000002
26 173.69
27 753.70999999999999
28 451.369999999999983
1 831.58999999999996
2 821.21999999999998
3 936.25999999999996
4 0.0
23 522.97999999999999
24 409.65999999999997
25 199.2
26 387.80999999999998
27 511.87999999999994
28 3216.3500000000001
29 162.23
30 159.37999999999994
31 156.93999999999997

```

In [36]: `#good dataframe`  
`good`

Out[36]:

|              | Timestamp        | Current | day | month | year | hour |
|--------------|------------------|---------|-----|-------|------|------|
| <b>27870</b> | 01-04-2019 00:00 | 0.0     | 1   | 4     | 2019 | 0    |
| <b>27871</b> | 01-04-2019 00:05 | 0.0     | 1   | 4     | 2019 | 0    |
| <b>27872</b> | 01-04-2019 00:10 | 0.0     | 1   | 4     | 2019 | 0    |
| <b>27873</b> | 01-04-2019 00:15 | 0.0     | 1   | 4     | 2019 | 0    |
| <b>27874</b> | 01-04-2019 00:20 | 0.0     | 1   | 4     | 2019 | 0    |
| ...          | ...              | ...     | ... | ...   | ...  | ...  |
| <b>2521</b>  | 31-12-2018 23:35 | 0.1     | 31  | 12    | 2018 | 23   |
| <b>2522</b>  | 31-12-2018 23:40 | 0.1     | 31  | 12    | 2018 | 23   |
| <b>2523</b>  | 31-12-2018 23:45 | 0.1     | 31  | 12    | 2018 | 23   |
| <b>2524</b>  | 31-12-2018 23:50 | 0.1     | 31  | 12    | 2018 | 23   |
| <b>2525</b>  | 31-12-2018 23:55 | 0.1     | 31  | 12    | 2018 | 23   |

134784 rows × 6 columns

In [19]: *#bad dataframe*  
bad

Out[19]:

|              | Timestamp        | Current | day | month | year | hour |
|--------------|------------------|---------|-----|-------|------|------|
| <b>29310</b> | 06-04-2019 00:00 | 0.10    | 6   | 4     | 2019 | 0    |
| <b>29311</b> | 06-04-2019 00:05 | 0.10    | 6   | 4     | 2019 | 0    |
| <b>29312</b> | 06-04-2019 00:10 | 0.10    | 6   | 4     | 2019 | 0    |
| <b>29313</b> | 06-04-2019 00:15 | 0.10    | 6   | 4     | 2019 | 0    |
| <b>29314</b> | 06-04-2019 00:20 | 0.10    | 6   | 4     | 2019 | 0    |
| ...          | ...              | ...     | ... | ...   | ...  | ...  |
| <b>1657</b>  | 28-12-2018 23:35 | 0.09    | 28  | 12    | 2018 | 23   |
| <b>1658</b>  | 28-12-2018 23:40 | 0.09    | 28  | 12    | 2018 | 23   |
| <b>1659</b>  | 28-12-2018 23:45 | 0.09    | 28  | 12    | 2018 | 23   |
| <b>1660</b>  | 28-12-2018 23:50 | 0.09    | 28  | 12    | 2018 | 23   |
| <b>1661</b>  | 28-12-2018 23:55 | 0.09    | 28  | 12    | 2018 | 23   |

44862 rows × 6 columns

In [20]: *#missing dataframe*  
missing

Out[20]:

|       | Timestamp        | Current | day | month | year | hour |
|-------|------------------|---------|-----|-------|------|------|
| 52062 | 24-06-2019 00:00 | 0.0     | 24  | 6     | 2019 | 0    |
| 52063 | 24-06-2019 00:05 | 0.0     | 24  | 6     | 2019 | 0    |
| 52064 | 24-06-2019 00:10 | 0.0     | 24  | 6     | 2019 | 0    |
| 52065 | 24-06-2019 00:15 | 0.0     | 24  | 6     | 2019 | 0    |
| 52066 | 24-06-2019 00:20 | 0.0     | 24  | 6     | 2019 | 0    |
| ...   | ...              | ...     | ... | ...   | ...  | ...  |
| 81721 | 04-10-2019 23:35 | 0.0     | 4   | 10    | 2019 | 23   |
| 81722 | 04-10-2019 23:40 | 0.0     | 4   | 10    | 2019 | 23   |
| 81723 | 04-10-2019 23:45 | 0.0     | 4   | 10    | 2019 | 23   |
| 81724 | 04-10-2019 23:50 | 0.0     | 4   | 10    | 2019 | 23   |
| 81725 | 04-10-2019 23:55 | 0.0     | 4   | 10    | 2019 | 23   |

11232 rows × 6 columns

In [38]:

# no of good days  
len(g)-1

Out[38]: 117

In [39]:

# no of bad days  
len(b)-1

Out[39]: 156

In [40]:

# no of missing days  
len(m)-1

Out[40]: 41

In [ ]: